CoE 163 Computing Algorithms and Architecture

Software Exercise 1

Analyzing and Profiling Algirthms

How many turns to unlock a combination lock?

Synopsis: Formulate and profile algorithms to compute the minimum number of turns needed to unlock some combination locks

Problem Statement

Combination locks are one of the easiest locks to pick without specialized equipment. These locks are unlocked when the dials are set to the unlock combination. Each dial contains digits from 0 to 9, and the numbers wrap around (i.e. incrementing from 9 will turn that dial to 0). Anyone can "brute-force" unlock them by trying out each and every combination possible.

You are working for a lock company and you are assigned to determine the minimum number of turns needed to unlock a combination lock given its initial and unlocked final value for quality control purposes. A "turn" is defined as an increment or decrement in value of a digit by one. For example, if the initial configuration of a lock shows 1239 and the unlock value is 2348, only four (4) turns are needed, like so

1 -> 2 : 1 turn
2 -> 3 : 1 turn
3 -> 4 : 1 turn
9 -> 8 : 1 turn

Since you expect to compute locks with an arbitrary number of combination dials, you decided to write a program that will compute the minimum number of turns needed to unlock a combination lock of N locks, given a starting point value.

Software Exercise Specs

Write a function in any high-level programming language that computes the minimum number of turns required to unlock a combination lock, for a given N and starting number combination. In a language named C, the function signature would look something like this:

```
int find_min_combi(int num_dials, int start, int end);
```

Input
Input to your program would be three numbers denoting the number of combination locks N, the initial configuration/combination of the lock S, and the unlock configuration E.

Output
Your program should output the minimum number of turns P needed to change the lock from its initial state to the final unlock configuration.

Constraints
$0 < N \leq 100$

Sample Inputs/Outputs

- Sample Input 1
  ```
  4 1239 2348
  ```
  Sample Output 1
  ```
  4
  ```
- Sample Input 2
  ```
  1 2 3
  ```
  Sample Output 2
  ```
  1
  ```
- Sample Input 2
  ```
  4 1234 9899
  ```
- Sample Output 2
  ```
  15
  ```

Profiling

Enclose your function call in a loop that runs it at least 1000 times, corresponding to processing 1000 combination locks. Remember to include a timer in your code so that you can measure the run time of your code. For example, in an old language named Turbo Pascal, this is how you may write your profiling algorithm.

```
program CombiLockTest;

var i: integer;

begin
  { TIMER start }
  for i:= 1 to 1000 do begin
     { COMBILOCK CALCULATOR here }
  end;
  { TIMER end }
end.
```

Use your profiling code to measure the **average run time**, **maximum runtime,** and **minimum runtime**. Since we would like to observe how your code is affected by increasing number of N, have two sets of test cases for your trials:
a) Locks are of the same length
b) Locks are of different (increasing) lengths

Assembly and Performance

Discuss with your group how you would rewrite the algorithm in assembly x86 or CPython bytecode and write an assembly version of your algorithm with the intent to optimize vs the original code, if possible. Some references are uploaded on UVLe. Assembly syntax is not too important, but make sure to capture how many instructions are needed for your algorithm.

After writing your assembly code, input your original (non-assembly) function into Compiler Explorer (https://godbolt.org/) and compare your assembly code to that generated by the compiler (select an optimization level to see how the assembly code compares with the one that you wrote).

Journal

Make sure to discuss the following in your journal for this software exercise.

1. **Algorithm:** Breakdown the problem and discuss what data structure(s) you deemed most appropriate for this problem. How did you analyze the problem? Is there a governing math formula to simplify your code? Did you apply any known algorithms to your solution?
2. **Analysis of your Algorithm:** Use asymptotic and/or amortized to discuss the complexity of your code
3. **Profiling:** Present your measurement data and compare your runtime results with the analysis of your algorithm. Discuss what takes up time in your algorithm and how the lock length and the starting position of the lock affects the runtimes
4. **Assembly and Performance:** How different or the same is your assembly code from that generated by the compiler? Relate the assembly code, (number of steps, branches, types of operations) with the runtime performance of your code